COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structure Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	July 28, 2026	**DATE SUBMITTED:**	August 4, 2026
NAME:	Jaime D.C. Godino Jr.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Array-Based vs. STL Stack Implementations

1. PROCEDURE OUTPUT

ILO A: Implement a stack using the C++ STL.

CODE

```cpp
#include<iostream>
#include <stack> //call the stack form the STL

main(){
    std::stack<int> newStack; //created an object

    std::cout<<"Is the stack empty? " << newStack.empty()<< std::endl;

    //insert a 3 elements:
    newStack.push(3);
    std::cout<<"Top of stack: " << newStack.top()<< std::endl;
    newStack.push(2);
    std::cout<<"Top of stack: " << newStack.top()<< std::endl;
    newStack.push(1);
    std::cout<<"Top of stack: " << newStack.top()<< std::endl;

    std::cout<<"Is the stack empty " << newStack.empty()<< std::endl;
    newStack.push(6);
    newStack.push(7);
    newStack.push(8);

    std::cout<<"Size of the stack is: " << newStack.size()<< std::endl;

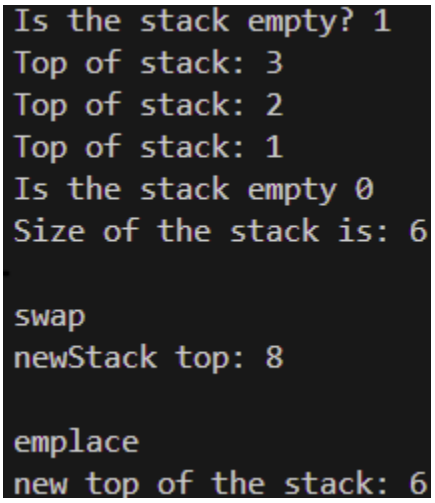
    //Swap?? -? swap the content of the public member functions
    std::cout<<"\nswap"<< std::endl;
    std::stack<int> secondStack;
    secondStack.push(9);
    secondStack.push(8);

    newStack.swap(secondStack);

    std::cout << "newStack top: " << newStack.top() << std::endl;
    //emplace -> construct and insert element -> public member function
    std::cout<<"\nemplace"<< std::endl;
    newStack.emplace(6);

    std::cout<<"new top of the stack: "<< newStack.top() << std::endl;
    return 0;
}
```

OUTPUT



OBSERVATION

This program starts with a stack by creating an empty stack. empty() checks whether it is empty, push() adds elements, top() displays the top element, size() determines the number of stored elements, swap exchanges the contents with another stack, and emplace inserts a new element efficiently.

ILO B.1: Implement a stack using a Static Array.

CODE

```
1   #include <iostream>
2
3   const size_t maxCap= 100;
4   int stack[maxCap];
5   int top = -1, newData, elements;
6
7   //prototyping
8
9   bool isEmpty();
10  bool isFull();
11  void push();
12  void pop();
13  void displayAll();
14  void Top();
15
```

```cpp
int main(){
    int choice;
    //asked the user for the maximum elements to be inserted
    std::cout<<"enter the number of maximum elements for the new stack (max: 100): ";
    std::cin>> elements;

    while(true){
        std::cout<<"Stack operations: \n";
        std::cout<<"1. PUSH \n2. POP \n3. Top \n4. isEmpty \n5. isFull \n6. displayAll" <<std::endl;
        std::cin >> choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout<< isEmpty()<< std::endl;
            break;
            case 5: std::cout<< isFull()<<std::endl;
            break;
            case 6: displayAll();
            break;
            default: std::cout<<"Invalid choice" <<std::endl;
            break;
        }
    }

    return 0;
}
```

```
47
48  bool isEmpty(){
49      //top is -1
50      if(top == -1 ) {
51          std::cout<<"Stack Underflow"<< std::endl;
52          return true;
53      }
54      return false;
55  }
56
57  bool isFull(){
58      //not equal to the max cap
59      if(top >= elements -1){
60          std::cout<<"Stack Overflow" << std::endl;
61          return true;
62      }
63      return false;
64  }
65
66  void push(){
67
68      //check if the stack is full, if yes return an error
69      if(isFull()) return;
70      //ask the user for a value to be pushed into the stack
71      std::cout<<"what number you want to add to the stack: ";
72      std::cin>> newData;
73      //store inside the stack?? pre or post increment
74      stack[++top] = newData;
75  }
76
77  void pop(){
78      //check if there are no more elements inside the stack
79      if(isEmpty()) return;
80
```

```
80
81      //display the element to be popped
82      std::cout<<"Popping: "<< stack[top]<<std::endl;
83
84      //decrement the top??
85      top--;
86
87  }
88
89  void Top(){
90      //error checking?
91      if(isEmpty()) return;
92
93      //display the top element
94      std::cout<<"The top element is: "<< stack[top]<<std::endl;
95
96  }
97
98  void displayAll(){
99      if(isEmpty()) return;
100
101     std::cout<<"Display all stack: "<< std::endl;
102
103     for(int elements = top; elements >= 0; elements--){
104         std::cout<<stack[elements]<<std::endl;
105     }
106 }
```

OUTPUT

PUSH

```
Stack operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
1
what number you want to add to the stack: 1
```

POP

```
Stack operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
2
Popping: 1
```

TOP

```
Stack operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
3
The top element is: 3
```

isEmpty

```
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
4
Stack Underflow
1
```

isFull

```
Stack operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
5
Stack Overflow
1
```

displayAll

```
Stack operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
6
Display all stack:
3
2
```

OBSERVATION

This program allows user to specify the max number of elements to be stored and has functions to push new values, pop, the top element, display the current top element using top(), check if whether the stack is empty or full using isEmpty() and isFull(), and display all elements from top to bottom. It also uses checking if the stack is overflowing or stack is underflowing.

ILOB B.2: Stacks using Linked Lists

CODE

```cpp
#include <iostream>
#include <iostream>
#include "listStack.h"

int main(){
    std::cout<<"------- Pushing 'I' 'E' 'E' 'E' ------ \n";

    //pushing
    push('I');
    push('E');
    push('E');
    push('E');

    //display top
    Top<char>();

    //display all elements in the stack
    std::cout<<"Display all: \n";
    displayAll<char>();

    //pop and print each element
    std::cout<<"------ popping 'E' 'E' 'E' 'I' ----- \n";
    while(!isEmpty<char>()){
        std::cout<<"popping: "<<pop<char>()<<std::endl;
    }
    //test stack underflow
    pop<char>();

    return 0;
}
```

listStack.h

```
#ifndef LIST_STACK_H
#define LIST_STACK_H

//class node with dat and next members
template <typename T>
class Node{
    public:
        T data;
        Node<T>* next = nullptr;
};

//create a pointer/reference tip that always point to current top

//global
template <typename T>
Node<T>* head= nullptr;

//pushing operation:
template <typename T>
void push(T newData){
    //create a new node
    Node<T>* newNode = new Node<T>;
    //assign the data to the new node
    newNode->data = newData;
    //link the new node to the current head
    newNode->next = head <T>;
    //make the new node the new top
    head<T> = newNode;
}

```

```
31  //isEmpty()
32  template <typename T>
33  bool isEmpty(){
34      return head<T> == nullptr;
35  }
36
37  //popping operation:
38  template <typename T>
39  T pop(){
40      //if the stack is empty error catch:
41      if(isEmpty<T>()){
42          std::cout<<"Stack underflow"<<std:: endl;
43
44      }
45      //saving a current top node to a temporary variable
46      Node<T>* temp = head<T>;
47      //store the data to a temporary variable
48      T tempVal = temp->data;
49      //move the head to next node
50      head<T> = head<T>->next;
51      //free up the memory
52      delete temp;
53      //return the temprary variable data
54      return tempVal;
55  }
56
57  template <typename T>
58  void Top(){
59      //what if the stack is empty
60      if(isEmpty<T>()) return;
```

```
62      std::cout<<"the top is: " << head<T> ->data << std::endl;
63  }
64
65  //display all
66  template <typename T>
67  void displayAll(){
68      //stack is empty
69      if (isEmpty<T>()) return;
70      //store the head with temp
71      Node<T>* temp = head<T>;
72      while (head<T> != nullptr){
73          std::cout<< head<T>->data <<std::endl;
74          head<T> = head<T>->next;
75
76      }
77      //ensure the head does not change its value
78      head<T> = temp;
79      std::cout << std::endl;
80  }
81
82  #endif
```

OUTPUT

```
------- Pushing 'I' 'E' 'E' 'E' ------
the top is: E
Display all:
E
E
E
I

------ popping 'E' 'E' 'E' 'I' -----
popping: E
popping: E
popping: E
popping: I
Stack underflow
```

OBSERVATION

This program implements a stack using a linked list, which allows elements to be added and removed dynamically without a fixed size limit. It supports the basic stack operations such as push, pop, top, isEmpty, and displayAll, while handling stack underflow. Comparing it to the array implementation, the linked list version is more flexible since it is more suitable for numbers of elements that are not known.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:
The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:

Problem Definition:
Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.

Steps:

1. Create a Stack.
2. While(end of input is not reached) {

a) If the character read is not a symbol to be balanced, ignore it.
b) If the character is an opening symbol, push it onto the stack.
c) If it is a closing symbol:
i) Report an error if the stack is empty.
ii) Otherwise, pop the stack.
d) If the symbol popped is not the corresponding opening symbol, report an error.

3. At the end of input, if the stack is not empty: report an error.

Self-Checking:

For the following cases, complete the table using the code you created.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)			
((A+B)+(C-D)			
((A+B)+[C-D])			
((A+B]+[C-D]}			

Stack using Array

mainArray.cpp

```cpp
#include <iostream>
#include <string>

const int maxCap = 100;
char stack[maxCap];
int top = -1;

void push(char c);
char pop();
char Top();
bool isFull();
bool isEmpty();
bool isMatching(char a, char b);
bool balanceCheck(const std::string& expression);

int main(){
    std::string inpExpression;
    std::cout<<"Balance Checker (Type stop to exit)" <<std::endl;

    while(true) {
        std::cout<<"Enter Expression: ";
        std::getline(std::cin, inpExpression);
        if (inpExpression == "" || inpExpression == "stop") break;

        top = -1;
        bool isValidExpression = balanceCheck(inpExpression);
        std::cout <<"Valid (Y/N) : " << (isValidExpression ? "Y" : "N") << std::endl;

    }
    return 0;
}
```

```
33  bool isMatching(char a, char b) {
34      if (a == '(' && b == ')') return true;
35      if (a == '{' && b == '}') return true;
36      if (a == '[' && b == ']') return true;
37      return false;
38  }
39
40  bool balanceCheck(const std::string& expression) {
41      for (char ch : expression) {
42          if (ch == '(' || ch == '{' || ch == '[') {
43              push(ch);
44          }
45          else if (ch == ')' || ch == '}' || ch == ']') {
46              if (isEmpty()) return false;
47              char topChar = pop();
48              if (!isMatching(topChar, ch)) {
49                  return false;
50              }
51          }
52      }
53      return isEmpty();
54  }
55
56  bool isEmpty(){
57      return top == -1;
58  }
59
60  bool isFull(){
61      return top >=maxCap -1;
62  }
63
64  void push(char c){
```

```
65      if (isFull()) {
66          std::cout << "Stack Overflow!\n";
67          return;
68      }
69      stack[++top] = c;
70  }
71
72  char pop(){
73      if(isEmpty()) return '\0';
74      return stack[top--];
75  }
76
77  char Top(){
78      if(isEmpty()) return '\0';
79      return stack[top];
80  }
```

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	Balance Checker (Type stop to exit) Enter Expression: (A+B)+(C-D) Valid (Y/N) : Y	The expression is valid because every opening parenthesis has a matching closing parenthesis in the correct order. The program pushes opening symbols and pops them when a matching closing symbol is found.
((A+B)+(C-D)	N	Enter Expression: ((A+B)+(C-D) Valid (Y/N) : N	The expression is invalid because one opening parenthesis is left unmatched. The stack

			still contains an opening parenthesis.
((A+B)+[C-D])	Y	Enter Expression: ((A+B)+[C-D]) Valid (Y/N) : Y	The expression is valid since both parenthesis and square brackets are matched.
((A+B]+[C-D]}	N	Enter Expression: ((A+B]+[C-D]} Valid (Y/N) : N	The expression is invalid because the closing symbols do not match the corresponding opening symbols.

Stack using Linked Lists

supplementalLinkedStack.h

```
#ifndef LIST_STACK_H
#define LIST_STACK_H

template <typename T>
class Node{
    public:
        T data;
        Node<T>* next = nullptr;
        Node(T val) : data(val), next(nullptr){}
    };

template <typename T>
class ListStack {
    private:
        Node<T>* head;
    public:
        ListStack() : head(nullptr){}

        ~ListStack() {
            clear();
        }

    bool isEmpty() const {
        return head == nullptr;
    }

    void push(T newData) {
        Node<T>* newNode = new Node<T>(newData);
        newNode->next = head;
        head = newNode;
    }

```

```
33      T pop() {
34          if (isEmpty()) {
35              std::cerr << "Stack Underflow" << std::endl;
36              return T();
37          }
38          Node<T>* temp = head;
39          T tempVal = temp->data;
40          head = head ->next;
41          delete temp;
42          return tempVal;
43      }
44
45      T peak () const {
46          if(!isEmpty()) return head-> data;
47          return T();
48      }
49
50      void clear() {
51          while(!isEmpty()) {
52              pop();
53          }
54      }
55  };
56
57
58  #endif
```

mainLinkedList.cpp

```cpp
#include <iostream>
#include <string>
#include "supplementalListStack.h"

bool isMatching(char a, char b);
bool balanceCheck(const std::string& expression);

int main() {
    std::string inpExpression;
    std::cout << "Balance Checker (Type stop to exit)" << std::endl;

    while(true) {
        std::cout<<"Enter expression: ";
        std::getline(std::cin, inpExpression);
        if (inpExpression == "" || inpExpression == "stop") break;

        bool isValidExpression = balanceCheck(inpExpression);
        std::cout << "Valid? (Y/N) : " << (isValidExpression ? "Y" : "N") << std::endl;
    }

    return 0;
}

bool isMatching(char a, char b) {
    if (a == '(' && b == ')') return true;
    if (a == '{' && b == '}') return true;
    if (a == '[' && b == ']') return true;
    return false;
}

bool balanceCheck(const std::string& expression) {
    ListStack<char> stack;
```

```
33      for (char ch : expression) {
34          if (ch == '(' || ch == '{' || ch == '[') {
35              stack.push(ch);
36          }else if (ch == ')' || ch == '}' || ch == ']') {
37              if (stack.isEmpty()) return false;
38              char topChar = stack.pop();
39              if (!isMatching(topChar, ch)) {
40                  return false;
41              }
42          }
43      }
44      return stack.isEmpty();
45  }
```

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis

(A+B)+(C-D)	Y	`Balance Checker (Type stop to exit)`
		`Enter expression: (A+B)+(C-D)`
		`Valid? (Y/N) : Y`
			This expression is valid, and every opening symbol is stored as a node in the linked list and removed once the matching closing symbol is found.

((A+B)+(C-D)	N	`Enter expression: ((A+B)+(C-D)` `Valid? (Y/N) : N`	This expression is invalid, and one node still remains in the linked list.
((A+B)+[C-D])	Y	`Enter expression: ((A+B)+[C-D])` `Valid? (Y/N) : Y`	The expression is valid, and the linked list stores each opening symbol and removes it only when the correct closing symbol is matched.
((A+B]+[C-D]}	N	`Enter expression: ((A+B]+[C-D]}` `Valid? (Y/N) : N`	This expression is invalid, and it detects mismatched delimiters, which cause the validation to fail.

Stack using C++ STL

mainSTL.cpp

```cpp
#include <iostream>
#include <stack>
#include <string>

bool isMatching(char a, char b);
bool balanceCheck(const std::string& expression);

int main() {
    std::string inpExpression;
    std::cout << "Balance Checker (Type stop to exit)" << std::endl;

    while(true) {
        std::cout<<"Enter expression: ";
        std::getline(std::cin, inpExpression);
        if (inpExpression == "" || inpExpression == "stop") break;

        bool isValidExpression = balanceCheck(inpExpression);
        std::cout << "Valid? (Y/N) : " << (isValidExpression ? "Y" : "N") << std::endl;
    }

    return 0;
}

bool isMatching(char a, char b) {
    if (a == '(' && b == ')') return true;
    if (a == '{' && b == '}') return true;
    if (a == '[' && b == ']') return true;
    return false;
}

bool balanceCheck(const std::string& expression)
{
```

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis

(A+B)+(C-D)	Y	`Balance Checker (Type stop to exit)`
		`Enter expression: (A+B)+(C-D)`
		`Valid? (Y/N) : Y`
			The STL stack pushes each opening symbol onto the stack and pops it when the corresponding closing symbol is matched.

((A+B)+(C-D)	N	Enter expression: ((A+B)+(C-D) Valid? (Y/N) : N	After read and writing all, an opening symbol still remains in the stack. Therefore it is unbalanced.
((A+B)+[C-D])	Y	Enter expression: ((A+B)+[C-D]) Valid? (Y/N) : Y	The STL stack stores each opening symbol and removes it only when matched to closing symbol.
((A+B]+[C-D]}	N	Enter expression: ((A+B]+[C-D]} Valid? (Y/N) : N	This is invalid because the STL stack detects mismatched pairs.

Tools Analysis:

● How do the different internal representations affect the implementation and usage of the stack?

The three stack implementation differ in how they store and manage data. The array-based stack is simple and fast but has a fixed size, while the linked list stack can grow dynamically as more elements are added. The C++ stack is the easiest to implement because it already has a built-in stack operations.

Conclusion

This activity helped me understand that stack operations can be implemented in different ways. I also learned the strength and limitations of array-based, linked list, and C++ STL implementations in terms of memory management and ease of implementation. Personally, I would use C++ STL as it is quite easy to use and understand. Overall, the activity improved my understanding of stacks and demonstrated how they can be applied identifying whether an expression is balanced or not.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I used ChatGPT and asked how would I implement the stack using C++ STL with the code I had made beforehand

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.